

DAIMO: un perfil de integración para activos de modelos de IA gobernados en espacios de datos

Edmundo de Elvira Mori Orrillo*

Jiayun Liu

Universidad Politécnica de Madrid, España

Resumen

Resumen. El despliegue interorganizacional de modelos de inteligencia artificial exige un tratamiento semántico coherente de cinco tareas que hoy están fragmentadas entre vocabularios distintos: publicar el modelo como activo de catálogo, seleccionarlo bajo una política de uso, invocarlo mediante un servicio tipado, trazar cada ejecución y compararlo sobre condiciones reproducibles. Los vocabularios existentes resuelven cada tarea por separado — MLDCAT-AP describe el modelo, DCAT lo publica, ODRL expresa la política, PROV-O registra la procedencia, y el Dataspace Protocol (DSP) negocia el intercambio contractual—, pero ninguna ontología articula hoy el puente entre ellos. Este artículo presenta DAIMO (Data-space AI Model Ontology), un perfil de integración OWL 2 DL que reutiliza los cinco vocabularios anteriores con alineamientos axiomáticos verificados e introduce las clases nativas necesarias para tratar los modelos de IA como activos de primera clase gobernados. La ontología se ha desarrollado siguiendo la metodología LOT y se acompaña de una capa SHACL con restricciones estructurales, invariantes SPARQL entre clases y un banco de pruebas negativas. DAIMO se ejercita sobre un escenario multi-participante —un modelo de predicción de riesgo de inundación intercambiado entre una universidad, un ayuntamiento y una plataforma federada de espacios de datos— que instancia las veintitrés preguntas de competencia elicítadas durante la especificación de requisitos. El artículo describe la justificación del diseño, los compromisos de alineamiento, la evidencia de corrección y las limitaciones conocidas del artefacto.

Palabras clave: ontología; perfil de integración; modelos de IA; espacios de datos; MLDCAT-AP; DCAT; ODRL; PROV-O; SHACL; LOT.

1. Introducción

El Reglamento (UE) 2024/1689, conocido como *AI Act*, exige para los sistemas de inteligencia artificial clasificados como de alto riesgo la documentación técnica, el registro de ejecuciones, la trazabilidad del entrenamiento y la evidencia de auditoría a lo largo de todo el ciclo de vida [37]. Paralelamente, la iniciativa europea de espacios de datos —materializada en los conectores Eclipse Dataspace Components (EDC) [35] y en despliegues sectoriales como INESData o Gaia-X— consolida un modelo de intercambio contractual entre participantes autónomos basado en el Dataspace Protocol (DSP) [34]. La conjunción de ambos marcos crea una demanda concreta: los modelos de inteligencia artificial deben poder circular entre organizaciones como activos gobernados, con metadatos accionables por máquina que soporten publicación en catálogo, descubrimiento sensible a política, invocación controlada mediante un servicio, trazabilidad de cada ejecución y comparación reproducible de resultados. Los vocabularios de la Web Semántica que abordan cada una de esas cinco tareas por separado están maduros, pero ninguno de ellos articula la cadena completa.

* Autor de correspondencia: `edmundo.mori.orrillo@upm.es`.

1.1. Motivación

MLDCAT-AP 3.0.0 (SEMIC, 2025) [33] extiende DCAT para describir modelos de aprendizaje automático como activos de catálogo, con metadatos ricos sobre tarea, algoritmo, ejecuciones, evaluaciones, riesgos y recursos computacionales. DCAT [2] aporta las primitivas de catálogo, distribución y servicio. ODRL [4] ofrece un modelo consolidado de ofertas, permisos, prohibiciones y acuerdos. PROV-O [3] es el estándar para procedencia y atribución. DSP [34] especifica los mensajes de negociación de contratos y transferencia de datos que EDC implementa. Artefactos de documentación como las *model cards* [6], *factsheets* [7] y *datasheets* [19] complementan estos vocabularios con descripciones narrativas orientadas al lector humano.

Cada uno de estos vocabularios resuelve bien su parte del problema pero ninguno resuelve el puente. Un modelo publicado como `it6:MachineLearningModel` en MLDCAT-AP no lleva adherido el acto de ser ofrecido a través de una frontera organizativa; una `odrl:Offer` no está anclada a un modelo concreto dentro del catálogo; una negociación DSP no sabe que el activo negociado es un modelo de IA con métricas y contextos de evaluación; y un `bundle PROV` no agrupa por sí solo las actividades ocurridas bajo contextos administrativos distintos. Publicar, descubrir, invocar, trazar y comparar modelos en un espacio de datos exige una capa semántica de integración que no existe hoy como ontología reutilizable.

1.2. Brecha de investigación

La brecha que aborda este trabajo se formula con precisión: *no existe todavía una ontología reutilizable que vincule metadatos de publicación en catálogo, políticas de acceso, contratos de invocación, trazas de ejecución, evidencia de auditoría y evaluación comparativa para modelos de IA que circulan entre participantes de un espacio de datos, con alineamientos axiomáticos verificados contra los vocabularios existentes y con evidencia ejecutable y reproducible de su corrección*. Cerrar esta brecha exige respetar dos principios simultáneamente: reutilizar lo que los vocabularios maduros ya resuelven, y añadir sólo los constructos que el puente entre ellos requiere.

1.3. Contribuciones

Este artículo presenta DAIMO (Data-space AI Model Ontology), diseñada específicamente para cerrar esa brecha, y hace cuatro contribuciones concretas.

La primera es el propio **perfil de integración DAIMO**, una ontología OWL 2 DL que reutiliza DCAT-AP, MLDCAT-AP 3.0.0, ODRL 2.2, PROV-O y DSP mediante alineamientos axiomáticos `rdfs:subClassOf` y `rdfs:subPropertyOf` —no listas de prefijos—, e introduce nueve clases nativas de primer nivel más cinco subclases de rol que cubren exactamente los constructos que la unión de los vocabularios reutilizados no proporciona. Cada clase nativa se justifica explícitamente por un hueco concreto y por al menos una pregunta de competencia que no puede responderse sin ella.

La segunda es una **verificación de implicación** sobre el cierre OWL-RL que complementa el razonamiento estándar de consistencia. Para cada clase nativa, la comprobación enumera las superclases inferidas y las contrasta con una lista de superclases prohibidas semánticamente. Este mecanismo detecta alineamientos que un razonador DL acepta sin contradicción pero que introducen tipados no intencionados —un tipo de error que afecta con frecuencia a las alineaciones hacia PROV-O y que, hasta donde conocemos, ninguno de los perfiles comparables en el SWJ reciente verifica explícitamente.

La tercera es una **capa SHACL con invariantes entre clases y banco de pruebas negativos**. DAIMO publica *shapes* estructurales de completitud por clase, *shapes* de conformidad sobre clases reutilizadas, y seis invariantes SHACL-SPARQL que codifican reglas de gobernanza expresables sólo sobre varias clases simultáneamente. Cada invariante se acompaña de un caso

de violación deliberado; la validación conforma sobre el grafo positivo y se dispara sobre el grafo negativo, demostrando que las restricciones son discriminantes y no meramente satisfacibles.

La cuarta es un **caso de uso multi-participante** que instancia la ontología sobre un escenario de predicción de riesgo de inundación intercambiado entre la Universidad Politécnica de Madrid, el Ayuntamiento de Leganés y la plataforma INESData. El caso actúa como demostrador: las instituciones nombradas son reales, los datos sintéticos. Ejerce cada clase nativa y hace que las veintitrés consultas de competencia devuelvan las filas esperadas sobre el cierre OWL-RL del grafo resultante.

1.4. Organización

La Sección 2 sitúa DAIMO frente a cuatro familias de trabajo relacionado. La Sección 3 describe la metodología LOT seguida y los requisitos elicitados. La Sección 4 presenta el perfil ontológico en detalle, incluyendo las decisiones de alineamiento cuya defensa explícita es central al argumento. La Sección 5 presenta el caso de estudio multi-participante e ilustra cómo cada tipo de pregunta de competencia se resuelve sobre él. La Sección 6 expone la evaluación y validación multicapa. La Sección 7 discute fortalezas, limitaciones y relación con trabajos adyacentes. La Sección 8 resume disponibilidad y sostenibilidad. La Sección 9 concluye.

2. Trabajo relacionado

El problema que DAIMO aborda se sitúa en la intersección de cuatro familias de trabajo que con frecuencia se tratan por separado. Esta sección las revisa por orden de proximidad al problema, destacando en cada caso qué aporta la familia, dónde deja abierto el puente y cómo DAIMO se posiciona respecto a ella. La Tabla 1 sintetiza la comparación.

2.1. Documentación de modelos

Las *model cards* [6], *factsheets* [7] y *datasheets for datasets* [19] han sido decisivas para elevar los estándares de transparencia en la publicación de artefactos de inteligencia artificial. Su contribución es principalmente documental: describen en lenguaje natural el uso previsto, las limitaciones, el rendimiento esperado y las condiciones bajo las que un modelo ha sido entrenado y evaluado. Esto las hace valiosas para la comunicación y para la rendición de cuentas cualitativa, pero débiles como sustrato para descubrimiento automatizado, filtrado por política o vinculación de evidencia en flujos máquina a máquina. Un espacio de datos requiere primitivas que los agentes de software puedan consumir; las descripciones narrativas son complementarias pero no suficientes.

2.2. Ontologías del ciclo de vida de aprendizaje automático

EXPO [24] ofrece una ontología general de experimentos científicos que ha servido de base a trabajos posteriores. ML-Schema [25] formaliza la semántica de experimentos de aprendizaje automático mediante un esquema ligero y reutilizable. MEX [26] provee un vocabulario de intercambio para resultados de experimentos, y OntoDM [27] junto con DMOP [28] articulan la minería de datos y el *semantic meta-mining*. Estos trabajos establecieron un fundamento riguroso pero permanecen centrados en el experimento científico más que en el activo desplegable. Souza [29] y Schmidt [30] extienden la perspectiva hacia la procedencia y la FAIRness del ciclo de vida ML, pero sin abordar la dimensión contractual entre participantes.

El trabajo más reciente y más próximo al dominio DAIMO es MLDCAT-AP 3.0.0 [33], publicado por SEMIC (European Commission, 2025). MLDCAT-AP extiende DCAT para describir modelos de IA como subclases de `dcat:Dataset` con metadatos ricos sobre política, benchmark, evaluación, ejecución, riesgo e infraestructura. Es fuerte en el lado del modelo —reutilizamos

directamente su clase `it6:MachineLearningModel` así como `it6:Task`, `it6:Run`, `it6:Flow`, `it6:Evaluation` e `it6:ComputerInfrastructure`— pero es deliberadamente agnóstico respecto al espacio de datos en el que el modelo circula: no modela el acto de *ofrecer* un modelo a través de fronteras organizativas, ni las nociones de despliegue gobernado, autorización de ejecución anclada a un acuerdo, o procedencia agrupada entre contextos administrativos distintos. MLDCAT-AP cubre el modelo; DAIMO cubre el puente entre el modelo y el dataspace.

2.3. Vocabularios de gobernanza

DCAT 2 [2] proporciona las primitivas de catálogo (`dcat:Catalog`, `dcat:Dataset`, `dcat:Distribution`, `dcat:DataService`, `dcat:CatalogRecord`) sobre las que se apoyan tanto MLDCAT-AP como DAIMO. ODRL 2.2 [4] ofrece un modelo consolidado para permisos, prohibiciones, obligaciones, ofertas (`odrl:Offer`) y acuerdos (`odrl:Agreement`), y es la base para expresar las políticas de uso que gobiernan la invocación del modelo. PROV-O [3] aporta la ontología de procedencia con actividades, entidades, agentes, bundles y roles, y es la capa natural sobre la que codificar la trazabilidad de ejecuciones. DPV y trabajos relacionados como Pandit y Esteves [15] complementan ODRL con semántica de privacidad. Estos vocabularios son maduros y ampliamente adoptados, pero por construcción son agnósticos al dominio: ninguno es específico de inteligencia artificial ni de espacios de datos.

2.4. Estándares de espacios de datos

La perspectiva de los espacios de datos desplaza el foco desde artefactos aislados hacia intercambios gobernados entre actores autónomos. Franklin, Halevy y Maier [1] formularon las *dataspaces* como respuesta a la heterogeneidad persistente de fuentes. Otto [17] y Cappiello [18] convierten la soberanía del dato, los contratos y la interoperabilidad controlada en supuestos explícitos del diseño. El Dataspace Protocol (DSP) [34] es la especificación W3C-CG que define los mensajes de catálogo, la negociación de contratos y los procesos de transferencia; Eclipse Dataspace Components [35] es su implementación de referencia.

Es crucial distinguir aquí entre *framework* y *vocabulario*: EDC es un framework de ejecución en Java, no una ontología. La semántica que EDC implementa proviene de DSP (protocolo), ODRL (políticas) y DCAT (catálogo); sólo un puñado de conceptos son verdaderas extensiones específicas de EDC, y de ellos DAIMO referencia explícitamente `edc:ParticipantContext` por su utilidad para anclar la procedencia agregada a un contexto administrativo. Esta aclaración es importante porque en la literatura de espacios de datos los dos términos se confunden con frecuencia. Trabajos sobre ecosistemas federados como ConSolid [8] muestran además que la Web Semántica evoluciona hacia escenarios multi-actor donde gobernanza e interoperabilidad deben diseñarse conjuntamente, no como capas independientes.

2.5. Ontologías recientes con validación ejecutable en el SWJ

El *Semantic Web Journal* ha publicado recientemente un conjunto de ontologías que comparten un compromiso metodológico explícito con la validación ejecutable. Kurteva et al. presentan RePlanIT, una ontología para pasaportes digitales de producto de equipos ICT, con SHACL y evaluación con expertos de industria [9]. Palma et al. operacionalizan información de suelos a escala global con GloSIS [10]. Woods et al. conectan ingeniería ontológica con procesamiento del lenguaje natural para actividades de mantenimiento industrial [11]. Bareedu et al. derivan reglas de validación SHACL desde estándares industriales OPC UA [13]. Ferranti et al. formalizan restricciones de propiedad de Wikidata mediante SHACL y SPARQL [14]. Robaldo y Batsakis analizan la interacción entre validación SHACL e inferencia sobre la Ontología del Tiempo [16]. Penteado et al. estudian metodologías de publicación de linked open government data [12]. Estos trabajos establecen colectivamente la expectativa de que una ontología con pretensiones opera-

Tabla 1: Familias de trabajo relacionado y brecha abordada por DAIMO.

Familia	Fortaleza	Limitación para gestión de modelos en espacios de datos	Respuesta de DAIMO
Documentación de modelos	Descripciones narrativas de uso previsto, riesgos y rendimiento	Débil vinculación accionable por máquina entre catálogos, servicios, políticas y evidencia	Convierte los metadatos operativos mínimos en recursos tipados y consultables
Ontologías del ciclo de vida de ML (incl. MLDCAT-AP 3.0.0)	Descripciones formales de algoritmos, ejecuciones, evaluaciones y riesgos del modelo	No modelan la oferta gobernada del modelo a través de fronteras organizativas	Añade un perfil orientado a espacios de datos sobre el modelo de MLDCAT-AP
Vocabularios de gobernanza (DCAT, ODRL, PROV-O)	Publicación, política y procedencia maduras, agnósticas al dominio	No son específicos de IA ni de espacios de datos	Los reutiliza con alineamientos axiomáticos verificados
Estándares de espacios de datos (DSP, EDC)	Negociación contractual y transferencia entre participantes	Agnósticos respecto al tipo de activo; EDC es framework, no vocabulario	Conecta el plano del dataspace con el catálogo de modelos de IA

cionales presente evidencia reproducible de su corrección, no sólo una descripción axiomática. DAIMO adopta esta expectativa como vinculante.

2.6. La brecha que cubre DAIMO

A partir de la comparación anterior, la brecha puede formularse con la precisión necesaria para distinguir la contribución de DAIMO de cualquier combinación lineal de los trabajos citados. DAIMO añade, sobre MLDCAT-AP y los vocabularios de gobernanza, exactamente los constructos necesarios para que un modelo de IA circule entre participantes autónomos de un espacio de datos como activo de primera clase: un registro de catálogo que porta una política ODRL coherente con el modelo registrado, un despliegue tipado que media entre el modelo y el servicio que lo expone, una autorización de ejecución anclada a un acuerdo DSP, y una agregación de procedencia que cruza contextos administrativos. Ninguno de los vocabularios reutilizados proporciona estos constructos, y ninguno de los trabajos SWJ comparables aborda este puente.

3. Metodología y requisitos

DAIMO se ha desarrollado siguiendo la metodología LOT (*Linked Open Terms*) [31], elegida por dos razones. La primera es su orientación industrial: LOT prescribe un flujo de trabajo ligero pero disciplinado, estructurado en cuatro fases —especificación de requisitos, implementación, publicación y mantenimiento— con artefactos entregables en cada una. La segunda es su tratamiento explícito de la reutilización como fase de primera clase, no como subproducto: LOT obliga a contabilizar qué se reutiliza antes de decidir qué se introduce, lo que encaja directamente con la disciplina *reuse-first* que la naturaleza del problema impone. La Tabla 2 mapea cada fase de LOT al artefacto de DAIMO y a la sección del presente artículo donde dicho artefacto se describe.

3.1. Escenario ejecutivo

La especificación de requisitos parte de un escenario ejecutivo que, posteriormente, actúa como caso de estudio (Sección 5). La Universidad Politécnica de Madrid (UPM), como proveedor

Tabla 2: Mapeo entre fases LOT y artefactos producidos durante el desarrollo de DAIMO.

Fase LOT	Sección	Artefactos
Fase 1 – Especificación de requisitos	§3.2	ORSD con 23 preguntas de competencia; requisitos no funcionales (OWL 2 DL, CC-BY 4.0, FAIR, SHACL)
Fase 2a – Conceptualización	§3.3	Tres decisiones centrales de diseño; arquitectura modular
Fase 2b – Reutilización	§4.6	<code>alignment.ttl</code> con alineamientos <code>rdfs:subClassOf</code> y <code>rdfs:subPropertyOf</code>
Fase 2c – Codificación	§4	<code>daimo-core.ttl</code> con axiomas de disyunción, propiedades funcionales, asimétricas e inversas
Fase 2d – Evaluación	§6	Scripts <code>validate.py</code> , <code>reasoner_check.py</code> , <code>oops_check.py</code> y <code>tests/negative_test.py</code>
Fase 3 – Publicación	§8	Documentación HTML (WIDOCO); negociación de contenido bajo <code>w3id.org</code>
Fase 4 – Mantenimiento	§8	<code>CHANGELOG.md</code> , <code>CONTRIBUTING.md</code> , política Sem-Ver y criterio de deprecación

de modelos, publica un modelo de predicción de riesgo de inundación a 24 horas y 100 m de resolución sobre cuencas ibéricas en un catálogo federado gestionado por la plataforma INES-Data. El Ayuntamiento de Leganés, como consumidor, necesita invocar el modelo para emitir avisos locales tras una tormenta extrema, pero no puede hacerlo sin antes descubrirlo bajo una política compatible con el uso público, verificar que satisface un umbral mínimo de calidad sobre un contexto de evaluación común, negociar un acuerdo que autorice sus ejecuciones concretas, e invocarlo a través de un endpoint autenticado. El equipo climático del CSIC, en un rol de evaluador, compara el modelo con una línea base sobre el mismo contexto de evaluación. Una oficina de cumplimiento alineada con Gaia-X, como actor de gobernanza, audita las ejecuciones y archiva la evidencia correspondiente.

Este escenario sintetiza las cinco tareas operativas que una ontología del puente debe soportar simultáneamente —publicación, descubrimiento, invocación, trazabilidad y evaluación— e involucra a cinco tipos de actor claramente diferenciados cuyas necesidades son complementarias pero no coinciden. Es, además, verosímil: no depende de tecnología hipotética y se puede instanciar con los vocabularios reutilizados más las extensiones que DAIMO introduce.

3.2. Especificación de requisitos

A partir del escenario anterior y de dos fuentes documentales complementarias —el dossier interno de MLDCAT-AP 3.0.0 y el análisis de las extensiones Eclipse EDC empleadas por INESData— se formularon veintitrés preguntas de competencia organizadas en cinco categorías por actor y etapa del ciclo de vida. La categoría R (Registro y publicación) cubre las cinco preguntas que un proveedor debe poder responder al publicar el modelo como activo gobernado con identidad, licencia, política y contrato de invocación. La categoría D (Descubrimiento y selección) cubre las cuatro preguntas que un consumidor hace para localizar un modelo adecuado por tarea, dominio, política y umbral de calidad. La categoría E (Ejecución y auditabilidad) agrupa las cinco preguntas que un operador y un actor de gobernanza plantean para invocar servicios, trazar ejecuciones y reconstruir evidencia. La categoría V (Evaluación y reproducibilidad) contiene las cinco preguntas que un evaluador formula para comparar resultados bajo condiciones homogéneas. Finalmente, la categoría G (puente de Gobernanza) cubre las cuatro preguntas multi-actor cuya respuesta exige exclusivamente semántica nativa DAIMO y que, como se argumenta en la Sección 7, no pueden responderse con los vocabularios reutilizados aisladamente. La Tabla 3 resume la correspondencia actor-CQ; el conjunto completo de las veintitrés preguntas en lenguaje natural aparece en el Apéndice A.

Tabla 3: Actores involucrados en el escenario ejecutivo y categorías de preguntas de competencia derivadas.

Actor	Categoría	Necesidad	CQs
Proveedor de modelos	R	Publicar un modelo como activo gobernado con identidad, licencia, política y contrato de invocación	CQ-R1–CQ-R5
Consumidor de modelos	D	Descubrir modelos por tarea, dominio, política y umbral de calidad	CQ-D1–CQ-D4
Operador de plataforma	E	Invocar servicios, trazar ejecuciones y reconstruir evidencia operativa	CQ-E1–CQ-E5
Evaluador	V	Comparar resultados en contexto compartido e inspeccionar reproducibilidad	CQ-V1–CQ-V5
Gobernanza (multi-actor)	G	Puentear oferta, despliegue, autorización y procedencia entre participantes	CQ-G1–CQ-G4

Once de las veintitrés preguntas exigen razonamiento explícito —rutas de subclase, cadenas de subpropiedad o agregación SPARQL—, mientras que las doce restantes son consultas directas sobre datos DAIMO nativos. Esta proporción es deliberada: un perfil que se apoyara exclusivamente en consultas directas no ejercitaría la parte del artefacto que hace valiosa la semántica OWL sobre un endpoint SPARQL puro, y un perfil que exigiera razonamiento para toda pregunta sería impracticable en operación. Los requisitos no funcionales complementan los funcionales y fijan: perfil OWL 2 DL (para mantener decidibilidad y compatibilidad con HermiT y Pellet), licencia CC-BY 4.0 para la ontología y Apache-2.0 para los scripts de validación, URIs persistentes bajo <https://w3id.org/pionera/daimo> con negociación de contenido hacia las cinco serializaciones estándar, validación SHACL accesible y, sobre todos ellos, un compromiso explícito de *reuse-first*: cada término introducido por DAIMO debe justificarse en la documentación explicando por qué DCAT, DCAT-AP, MLDCAT-AP, ODRL, PROV-O, DSP o la extensión EDC no lo cubren.

3.3. Decisiones centrales de diseño

Tres decisiones estructurales se tomaron en la fase de conceptualización y condicionan todo el resto del diseño.

Decisión 1: reutilización preferente. DAIMO reutiliza directamente las clases de MLDCAT-AP para todo aquello que MLDCAT-AP ya modela con precisión. `it6:MachineLearningModel` es la clase del modelo; `it6:ComputerInfrastructure` describe el entorno de ejecución; `it6:Run`, `it6:Flow` y `it6:Evaluation` capturan respectivamente la ejecución concreta, el pipeline asociado y la medición resultante; `dcat:Dataset`, `dcat:Distribution` y `dcat:DataService` publican el activo; `odrl:Offer` y `odrl:Agreement` expresan la política y el acuerdo. Introducir equivalentes DAIMO-nativos de cualquiera de estas clases violaría el principio de compromiso ontológico mínimo de Gruber y fracturaría el ecosistema DCAT-AP sin ganancia; una clase DAIMO nativa sólo se justifica cuando la unión de los vocabularios reutilizados deja un hueco concreto que impide responder al menos una pregunta de competencia.

Decisión 2: separación en tres capas complementarias. La arquitectura distingue tres capas que DAIMO conecta pero no sustituye. La capa A es el plano del espacio de datos: DSP con las extensiones EDC puntuales que DAIMO referencia. La capa B es el plano de gobernanza de plataforma: INESData (o una plataforma análoga) con sus mecanismos de registro federado y políticas operativas. La capa C es el plano del activo de IA/ML: MLDCAT-AP y los vocabularios

que lo soportan. DAIMO es el *perfil de integración* que conecta A, B y C; no es una cuarta capa paralela que replique funcionalidad, sino un puente que hace consultable la articulación.

Decisión 3: una clase nativa por hueco, no por tema. Dada la reutilización preferente, cada clase DAIMO-nativa debe justificar su existencia por un hueco concreto en la unión de los vocabularios reutilizados y por al menos una pregunta de competencia que no puede responderse sin ella. El resultado son nueve clases de primer nivel —`AIAssetOffering`, `ParticipantRole`, `ModelDeployment`, `IOContract`, `ExecutionAuthorization`, `DerivedArtifact`, `CrossParticipantProvenance`, `AuditEvidence` y `SharedEvaluationContext`— y cinco subclases de rol que se describen en la Sección 4. Cualquier clase que no cumpla simultáneamente las dos condiciones del criterio se descarta.

4. La ontología DAIMO

4.1. Alcance y arquitectura modular

DAIMO modela el ciclo de vida operacional de un activo de modelo de IA dentro de un espacio de datos gobernado: publicación como registro de catálogo, descubrimiento sensible a política y a calidad, invocación controlada mediante servicios tipados, trazabilidad de ejecuciones individuales y evaluación contextualizada y reproducible. El alcance se restringe deliberadamente a esta cadena operacional porque las responsabilidades ortogonales —la modelización detallada del *pipeline* de entrenamiento, la semántica fina de políticas de privacidad, o la teoría formal de la derivación PROV— ya están cubiertas por vocabularios consolidados cuya duplicación sería contraproducente. DAIMO aporta, en cambio, los constructos necesarios para que esos vocabularios se articulen coherentemente en el caso concreto de un modelo de IA que circula entre participantes autónomos.

La ontología se estructura en tres módulos complementarios, distribuidos en archivos Turtle separados para que un consumidor pueda cargar únicamente los que necesite. El **módulo núcleo** (`daimo-core.ttl`) contiene las clases y propiedades DAIMO-nativas, los axiomas globales de disyunción y las caracterizaciones sobre propiedades (funcionales, asimétricas, inversas) que capturan las invariantes de tipo a nivel TBox. El **módulo de alineamientos** (`alignment.ttl`) encapsula las declaraciones `rdfs:subClassOf` y `rdfs:subPropertyOf` que conectan DAIMO con los vocabularios reutilizados, más las declaraciones mínimas de los términos externos que DAIMO referencia, cada una con un `rdfs:seeAlso` a la sección correspondiente de la especificación fuente. El **módulo SHACL** (`daimo-shapes.ttl`) aporta las restricciones estructurales por clase, las *shapes* de conformidad sobre clases reutilizadas y las invariantes cruzadas codificadas mediante `sh:SPARQLConstraint`; este módulo se declara a su vez como `owl:Ontology` independiente con su propia `owl:versionIRI`, lo que permite que la capa de validación se libere y se referencie como un artefacto con ciclo de vida propio.

La separación en tres módulos no es accidental. El núcleo puede ser consumido por una aplicación que sólo necesite las primitivas DAIMO; el módulo de alineamientos se carga cuando se quiere que un razonador infiera las relaciones entre DAIMO y los vocabularios reutilizados; el módulo SHACL se carga cuando se quiere validar un grafo concreto. La arquitectura modular se ilustra en la Figura 1.

4.2. Métricas del artefacto

La Tabla 4 resume las métricas de tamaño y expresividad de la ontología. Se reportan tanto el perfil DL declarado como los conteos exactos de clases nativas, propiedades y axiomas introducidos por DAIMO, junto con el tamaño de la capa SHACL. Las métricas permiten a un lector interesado estimar rápidamente el peso del artefacto antes de consumirlo y facilitan la comparación con ontologías publicadas previamente.

[PENDIENTE: Figura 1 (Chowlk): diagrama de la arquitectura modular de DAIMO. Tres rectángulos apilados (núcleo, alineamientos, SHACL) con flechas hacia las cinco capas de vocabularios reutilizados (DCAT, MLDCAT-AP, ODRL, PROV-O, DSP/EDC). El módulo de alineamientos debe aparecer como nodo intermedio conectando DAIMO con cada vocabulario externo. Generar con chowlk a partir del diagrama draw.io figures/architecture.drawio.]

Figura 1: Arquitectura modular de DAIMO y vocabularios reutilizados.

4.3. Vocabulario nativo

Las nueve clases nativas de primer nivel, más las cinco subclases de `ParticipantRole`, constituyen el conjunto mínimo que las tres decisiones centrales de diseño producen. La Tabla 5 resume cada una junto con su alineamiento axiomático a los vocabularios reutilizados y su papel específico.

Las dos clases sin alineamiento externo —`IOContract` y `SharedEvaluationContext`— lo son deliberadamente. `IOContract` no es un `dcatalog:DataService`: un servicio es un recurso accesible (una URL, un endpoint), mientras que el contrato describe las expectativas sintácticas y de autenticación sobre el intercambio. Tampoco es un `dct:Standard`: el contrato enumera obligaciones concretas, no refiere a una especificación externa. `SharedEvaluationContext` no es un `it6:Task` porque es una reificación metodológica (la combinación exacta bajo la que dos evaluaciones son comparables), no la tarea abstracta que las evaluaciones instancian. Alinearlos a clases genéricas como `owl:Thing` o `prov:Entity` añadiría ruido sin aportar semántica.

Para ilustrar la forma concreta que adopta una clase DAIMO-nativa en el archivo Turtle, el Listado 1 reproduce la declaración de `AIAssetOffering` en el núcleo de la ontología. La clase se alinea explícitamente a `dcatalog:CatalogRecord` mediante `rdfs:subClassOf`, lo que permite que cualquier motor compatible con DCAT reconozca la oferta como un registro de catálogo estándar y reutilice las herramientas existentes de descubrimiento. La propiedad asociada `daimo:offersModel` incorpora simultáneamente tres características semánticas: es funcional (una oferta registra exactamente un modelo), es asimétrica (una oferta nunca puede ser el modelo que ofrece) y se alinea con `foaf:primaryTopic`, que es la propiedad que DCAT recomienda para enlazar un registro de catálogo con el recurso que describe. El conjunto de declaraciones permite que tanto razonadores OWL como motores SPARQL exploten la relación sin conocer el vocabulario DAIMO-nativo específico.

Listing 1: Declaración Turtle de `daimo:AIAssetOffering` en el núcleo de DAIMO.

```
daimo:AIAssetOffering a owl:Class ;
  rdfs:subClassOf dcatalog:CatalogRecord ;
  skos:definition "A catalog-record entry registering an AI model in a
    dataspace catalog as a governed offer..."@en ;
  skos:example "A research organisation publishing one of its
    machine-learning models into a dataspace catalog declares an
    AIAssetOffering that references the model, the organisation, and
    an ODRL offer policy."@en ;
  rdfs:comment "..."@en .

daimo:offersModel a owl:ObjectProperty ,
  owl:FunctionalProperty ,
  owl:AsymmetricProperty ;
  rdfs:subPropertyOf foaf:primaryTopic ;
  rdfs:domain daimo:AIAssetOffering ;
  rdfs:range it6:MachineLearningModel .
```

La Figura 2 muestra las nueve clases de primer nivel, sus alineamientos y las propiedades nativas que las conectan entre sí.

Tabla 4: Métricas del artefacto DAIMO.

Dimensión	Valor
Perfil lógico declarado	OWL 2 DL
Clases nativas de primer nivel	9
Subclases nativas (todas bajo <code>ParticipantRole</code>)	5
Clases reutilizadas referenciadas por nombre	20+ (DCAT, MLDCAT-AP, ODRL, PROV-O, SPDX, FOAF)
Propiedades nativas	29 (21 <code>owl:ObjectProperty</code> , 8 <code>owl:DatatypeProperty</code>)
Propiedades funcionales (<code>owl:FunctionalProperty</code>)	19
Propiedades asimétricas (<code>owl:AsymmetricProperty</code>)	5
Pares inversos explícitos (<code>owl:inverseOf</code>)	5
Alineamientos <code>rdfs:subClassOf</code> a vocabularios externos	7
Alineamientos <code>rdfs:subPropertyOf</code> a vocabularios externos	9
Axiomas de disyunción nombrados	1 (<code>daimo:TopLevelKindsDisjointness</code>)
<i>Shapes</i> SHACL nodales de completitud	9
<i>Shapes</i> SHACL de conformidad sobre clases reutilizadas	3
Invariantes SHACL-SPARQL entre clases (<code>sh:SPARQLConstraint</code>)	6
Preguntas de competencia con SPARQL	23
OOPS! — Pitfalls Críticos / Importantes / Menores	0 / 0 / 2

4.4. Defensa de los dos alineamientos de mayor carga semántica

Dos alineamientos de los siete `rdfs:subClassOf` declarados por DAIMO hacia vocabularios externos llevan más carga semántica que el resto, y merecen defensa explícita porque las contra-lecturas son plausibles y un lector familiarizado con DCAT y ODRL las planteará inmediatamente.

AIAssetOffering $\sqsubseteq$ dcat:CatalogRecord, no dcat:Dataset. DCAT distingue con cuidado entre el recurso ofrecido (el `dcat:Dataset`, que es la cosa sobre la que hay interés) y el registro que describe dicho recurso en un catálogo concreto (el `dcat:CatalogRecord`, que es el artefacto de metadatos). DAIMO alinea `AIAssetOffering` a `dcat:CatalogRecord` porque la oferta es conceptualmente el *registro de catalogación* del modelo en un espacio de datos concreto, con una política y un conjunto de metadatos específicos de ese contexto, no el modelo en sí —que es reutilizado directamente como `it6:MachineLearningModel`. La contra-lectura que subsumiría `AIAssetOffering` a `dcat:Dataset` conlleva dos cosas distintas: el modelo (que puede aparecer en varios catálogos con condiciones distintas) y su oferta en un catálogo específico (que es una, y con una política propia). Esta distinción es importante operacionalmente: una misma `it6:MachineLearningModel` puede ser referenciada por varias `AIAssetOfferings` con políticas distintas, y la propiedad `daimo:offersModel` (alineada a `foaf:primaryTopic`) captura que la oferta *trata sobre* el modelo, no que es el modelo.

ExecutionAuthorization $\sqsubseteq$ odrl:Agreement, no prov:wasDerivedFrom odrl:Agreement. La autorización de ejecución es el resultado específico de una negociación DSP que vincula per-

Tabla 5: Clases nativas de DAIMO con sus alineamientos axiomáticos a los vocabularios reutilizados.

Clase	Alineamiento	Papel
daimo:AIAssetOffering	dcat:CatalogRecord	Entrada de catálogo que registra un modelo de IA como oferta gobernada en un espacio de datos; porta una política ODRL mediante daimo:hasOfferPolicy.
daimo:ParticipantRole	prov:Role	Rol asumido por un participante respecto a un activo de IA. Cinco subclases: ModelProvider, ModelConsumer, PlatformOperator, Evaluator, GovernanceActor. Deliberadamente no disyuntas entre sí.
daimo:ModelDeployment	prov:Entity	Instancia en ejecución o alojada de un modelo, expuesta como dcat:DataService sobre una it6:ComputerInfrastructure.
daimo:IOContract	(sin alineamiento externo)	Contrato mínimo accionable por máquina: formato de entrada, formato de salida, método de autenticación y esquemas opcionales.
daimo:ExecutionAuthorization	odrl:Agreement	Acuerdo ODRL producido por una negociación DSP que autoriza ejecuciones concretas mediante daimo:authorizesRun; porta al menos un odrl:permission.
daimo:DerivedArtifact	prov:Entity, dcat:Resource	Salida gobernada y describable en catálogo producida por una ejecución bajo una autorización.
daimo:CrossParticipantProperty	prov:wasDerivedFrom	Agregación de prov:Activity y prov:Entity a través de contextos de participantes para formar una narrativa de auditoría.
daimo:AuditEvidence	prov:Entity	Registro de evidencia orientado a cumplimiento, con hash de integridad (spdx:Checksum estructurado), firmante y marca temporal.
daimo:SharedEvaluationContext	(sin alineamiento externo)	Reificación de las condiciones de comparabilidad: tarea, dataset, versión, protocolo y semilla aleatoria.

misos ODRL concretos a invocaciones concretas, y ODRL modela precisamente este tipo de instrumento vinculante como `odrl:Agreement` —el equivalente a un contrato suscrito entre un asignador y un asignatario, con permisos y restricciones efectivas. La contra-lectura que relacionaría la autorización con el acuerdo mediante `prov:wasDerivedFrom` trataría la autorización como un artefacto distinto del acuerdo, introduciendo una distinción que ODRL no necesita: en ODRL un `Agreement` ya es la entidad vinculante. La subsunción aprovecha toda la semántica ODRL sobre permisos, prohibiciones y obligaciones sin redefinirla, y permite que herramientas ODRL existentes traten las autorizaciones DAIMO como acuerdos estándar. Lo que DAIMO añade sobre `odrl:Agreement` es la propiedad `daimo:authorizesRun`, asimétrica, que vincula el acuerdo a las ejecuciones `it6:Run` que cubre —una relación que ODRL no prescribe pero que el puente al plano PROV requiere.

4.5. Infraestructura axiomática

La taxonomía es sólo el andamiaje visible de DAIMO; detrás de ella se encuentra una infraestructura axiomática que fija el significado de las clases y evita que un grafo sintácticamente correcto resulte semánticamente ambiguo. Esta infraestructura se organiza en tres estratos complementarios: axiomas OWL sobre las clases y propiedades nativas, restricciones SHACL nodales que especifican la forma esperada de cada instancia, e invariantes SHACL-SPARQL que cruzan varias clases y codifican reglas de negocio no expresables mediante *shapes* de completitud.

En el estrato OWL, la disyunción entre los nueve tipos nativos de primer nivel se declara de

[PENDIENTE: Figura 2 (Chowlk): diagrama UML/Chowlk de las nueve clases DAIMO-nativas más las cinco subclases de `ParticipantRole`, con flechas hacia sus superclases externas (`dcatalog:CatalogRecord`, `prov:Role`, `prov:Entity`, `odrl:Agreement`, `prov:Bundle`, `dcatalog:Resource`) y con las propiedades nativas principales (`offersModel`, `deploysModel`, `exposedAs`, `hasIOContract`, `authorizesRun`, `derivedFromRun`, `evidenceOf`, `records`) rotuladas. Generar con chowlk a partir de `figures/daimo-classes.drawio`.]

Figura 2: Clases nativas de DAIMO, alineamientos y propiedades principales.

forma explícita mediante el axioma nombrado `daimo:TopLevelKindsDisjointness`, instancia de `owl:AllDisjointClasses`. La decisión de convertir la disyunción en una entidad nombrada, en lugar de emitir pares anónimos `owl:disjointWith`, persigue tres beneficios prácticos. El primero es que la disyunción puede referenciarse en documentación y reportes de validación. El segundo es que un razonador puede citarla como causa específica cuando detecta una contradicción, facilitando el diagnóstico. El tercero es que la disyunción entre las cinco subclases de `ParticipantRole` queda deliberadamente fuera del axioma: un mismo agente jurídico puede asumir varios roles simultáneamente (p. ej. proveedor y operador) sin generar inconsistencia lógica, reflejando la realidad de que los roles son propiedades contextuales del agente, no clases exclusivas.

Al núcleo de disyunciones se añaden caracterizaciones sobre las propiedades DAIMO. Diecinueve propiedades se declaran funcionales (`owl:FunctionalProperty`) cuando su dominio es una clase con *shape* nodal, de modo que la cardinalidad máxima está axiomatizada simultáneamente en la capa lógica y en la capa SHACL correspondiente. Cinco propiedades clave —`offersModel`, `deploysModel`, `authorizesRun`, `derivedFromRun` y `evidenceOf`— se marcan como asimétricas, capturando formalmente que un despliegue no es el modelo que despliega, que una autorización no coincide con la ejecución que autoriza, y así sucesivamente. Finalmente, cinco pares inversas explícitas (`authorizedBy` ↔ `authorizesRun`, `hasDeployment` ↔ `deploysModel`, `hasDerivedArtifact` ↔ `derivedFromRun`, `hasAuditEvidence` ↔ `evidenceOf`, `hasOffering` ↔ `offersModel`) habilitan consultas inversas nativas sin obligar al publicador del grafo a materializar ambos sentidos.

El estrato SHACL se divide a su vez en dos familias claramente diferenciadas. En primer lugar, cada clase DAIMO-nativa lleva asociada una *shape* nodal de completitud que prescribe qué propiedades son obligatorias en una instancia bien formada; estas nueve *shapes* son la garantía sintáctica de que un publicador ha suministrado todo lo necesario para que las invariantes semánticas tengan sustrato sobre el que operar. En segundo lugar, tres *shapes* de *conformidad* —aplicadas a `odrl:Offer`, `it6:MachineLearningModel` e `it6:Run`— establecen qué propiedades de clases reutilizadas son obligatorias cuando un grafo declara adherirse al perfil DAIMO. La distinción entre ambas familias es deliberada: DAIMO no sobrescribe la ontología subyacente —eso sería fragmentar el ecosistema— sino que declara su compromiso con un subconjunto concreto de propiedades sobre clases externas, haciendo explícito el criterio de adhesión sin modificar las definiciones originales.

Por encima de estas *shapes* se sitúan las seis invariantes SHACL-SPARQL entre clases (INV-1 a INV-6), que se examinan en detalle en la Sección 6.2. Son el estrato que distingue a DAIMO de un simple perfil descriptivo: codifican reglas del tipo “toda oferta publicada debe exponer un modelo cubierto por la política ODRL asociada” o “toda ejecución referenciada por un artefacto derivado debe contar con una autorización vigente que la cubra”. El sentido de estas reglas no puede expresarse como restricción local a una sola *shape* porque involucran la coherencia entre varias clases simultáneamente. El Listado 2 muestra cómo se codifican los axiomas OWL para el caso concreto de `ExecutionAuthorization`, combinando la subclase con `odrl:Agreement`, la asimetría de `authorizesRun`, la funcionalidad de su inversa y la pertenencia al axioma nombrado de disyunción.

Listing 2: Axiomas centrales alrededor de `ExecutionAuthorization`.

```
daimo:ExecutionAuthorization a owl:Class ;
  rdfs:subClassOf odrl:Agreement ;
  skos:definition "An ODRL agreement produced by a DSP contract
    negotiation that binds specific permissions to run invocations..."@en .

daimo:authorizesRun a owl:ObjectProperty , owl:AsymmetricProperty ;
  rdfs:domain daimo:ExecutionAuthorization ;
  rdfs:range it6:Run .

daimo:authorizedBy a owl:ObjectProperty , owl:FunctionalProperty ;
  owl:inverseOf daimo:authorizesRun ;
  rdfs:domain it6:Run ;
  rdfs:range daimo:ExecutionAuthorization .

daimo:TopLevelKindsDisjointness a owl:AllDisjointClasses ;
  owl:members ( daimo:AIAssetOffering
    daimo:ExecutionAuthorization
    daimo:ModelDeployment
    daimo:DerivedArtifact
    daimo:IOContract
    daimo:AuditEvidence
    daimo:CrossParticipantProvenanceRecord
    daimo:SharedEvaluationContext
    daimo:ParticipantRole ) .
```

4.6. Reutilización y alineamientos

La Tabla 6 resume los vocabularios reutilizados y el uso concreto de cada uno. Tres observaciones son importantes. La primera es que las alineaciones hacia DCAT y ODRL son con los vocabularios estándar, no con entidades internas de ninguna implementación: este punto es relevante porque en la práctica de Eclipse EDC las dos capas a menudo se confunden. La segunda es que DAIMO mantiene hacia DSP una relación de mapeo informativo mediante `skos:closeMatch` y `skos:related`, no de subsunción: DSP es un protocolo de mensajería, no una ontología de activos, y subsumir clases DAIMO bajo constructos DSP introduciría tipados espurios. La tercera es que de toda la extensión EDC sólo un término aparece explícitamente en DAIMO, `edc:ParticipantContext`, que se referencia por su utilidad para anclar la procedencia agregada de un participante.

Tres subpropiedades que podrían parecer naturales *no* se declaran, y la razón merece documentación explícita. `daimo:authorizesRun` no se declara subpropiedad de `prov:used`, `daimo:grantedTo` no se declara subpropiedad de `prov:qualifiedAssociation`, y `daimo:evidenceOf` no se declara subpropiedad de `prov:hadActivity`. En los tres casos, el alineamiento aparentemente intuitivo produciría, bajo razonamiento RDFS u OWL-RL, que las autorizaciones se tipasen como actividades PROV, los agentes receptores como asociaciones reificadas y las evidencias como objetos de influencia —tipados que no sólo no son deseados sino que contradicen el modelo conceptual del puente DAIMO. La verificación de implicación (§6.1) se diseñó precisamente para detectar errores de este tipo antes de que contaminen el cierre inferido.

5. Caso de estudio: intercambio gobernado en INESData

Esta sección retoma el escenario presentado en la Sección 3.1 y lo instancia sobre DAIMO como caso de estudio. El grafo resultante contiene 209 tripletas que ejercen al menos una vez cada clase nativa de la ontología y sobre las que se ejecutan las veintitrés consultas de competencia reportadas en la evaluación. Las instituciones nombradas —UPM, Ayuntamiento de

Tabla 6: Vocabularios reutilizados por DAIMO y uso concreto de cada uno.

Vocabulario	Términos reutilizados	Uso en DAIMO
MLDCAT-AP 3.0.0 (it6:)	MachineLearningModel, Task, Run, Flow, Evaluation, EvaluationMeasure, Benchmark, ComputerInfrastructure, Hardware, Library, HarmRisk, Modality, trainedOn, testedOn	Capa del activo de IA; DAIMO no redefine el modelo.
DCAT 2 (dcat:)	Catalog, Dataset, Distribution, DataService, CatalogRecord, endpointURL	Publicación y descubrimiento.
ODRL 2.2 (odrl:)	Offer, Agreement, Permission, Prohibition, hasPolicy, target, assigner, assignee	Política, oferta y acuerdo.
PROV-O (prov:)	Activity, Entity, Agent, Bundle, Role, wasGeneratedBy, wasAssociatedWith	Procedencia y roles.
SPDX (spdx:)	Checksum, algorithm, checksumValue	Integridad de evidencia de auditoría.
DSP (dsp:)	ContractOffer, ContractNegotiation, TransferProcess	Semántica del plano del dataspace, mediante mapeos informativos skos:closeMatch / skos:related (no subsunción).
Extensión EDC (edc:)	ParticipantContext	Única extensión específica de EDC referenciada.

Leganés, CSIC, INESData y una oficina de cumplimiento alineada con Gaia-X— son reales; los identificadores de modelo, valores de métrica y versiones de dataset son sintéticos. El caso se declara explícitamente como *demostrador*: su propósito es mostrar que DAIMO puede instanciarse coherentemente sobre un escenario multi-participante verosímil y responder las preguntas de competencia formuladas, no proporcionar evidencia empírica sobre datos operativos reales. La instanciación contra un catálogo MLDCAT-AP operativo es una línea de trabajo explícita en §7 y se marca con **[PENDIENTE: Instanciar DAIMO sobre un extracto real de un catálogo MLDCAT-AP, o al menos obtener una carta de compromiso de INESData/UPM confirmando el uso de las instituciones nombradas en este escenario.]**

5.1. Composición del grafo de demostración

La UPM publica, como proveedor de modelos, tres instancias de `AIAssetOffering` dentro del catálogo `upm-catalog` (un `dcat:Catalog` con propiedades `dcat:record` hacia las tres ofertas). La primera, `offering-flood-v2`, registra el modelo `flood-risk-v2`; la segunda, `offering-flood-v1-baseline`, registra la línea base `flood-risk-v1`; la tercera, `offering-flood-v2-commercial`, registra una variante con política comercial del mismo modelo v2. Las tres ofertas comparten la misma `it6:Task` (`Flood risk prediction`) y dominio (`Climate analytics`), pero difieren en política ODRL: la oferta de baseline y la de v2 llevan una política permisiva con permiso `odrl:use`, mientras que la tercera oferta añade una `odrl:prohibition` sobre la acción `odrl:commercialize`.

El modelo `flood-risk-v2` se despliega (mediante una instancia de `ModelDeployment`) sobre `upm-gpu-cluster`, una `it6:ComputerInfrastructure` descrita con PyTorch 2.5, CUDA 12.2 y cuatro GPUs A100. El despliegue se expone como *dos* `dcat:DataServices` simultáneos con sendos `IOContracts`: un endpoint REST en <https://api.inesdata.example.org/flood-risk/v2> con formato de entrada JSON, salida GeoJSON y autenticación `oauth2-bearer`, y un endpoint gRPC en <https://grpc.inesdata.example.org/flood-risk/v2> con protobuf y auten-

ticación `mtls`. Este uso multi-endpoint ejercita explícitamente la decisión de modelado de tratar `daimo:exposedAs` y `daimo:hasIOContract` como propiedades no funcionales: un mismo despliegue puede anunciar varios servicios y contratos de entrada/salida simultáneamente, cubriendo protocolos y perfiles de autenticación distintos.

El Ayuntamiento de Leganés, como consumidor, obtiene una instancia de `ExecutionAuthorization` ligada a un permiso `odrl:use` concreto, con fechas de vigencia explícitas. La ejecución `run-2026-04-20-legs` es una `it6:Run` que `prov:wasAssociatedWith` el Ayuntamiento y que está autorizada a través de `daimo:authorizesRun`. La ejecución produce un `DerivedArtifact` —la predicción georreferenciada de riesgo de inundación por sección censal— y un `AuditEvidence` cuyo `daimo:integrityHash` apunta a un `spx:Checksum` nombrado (`ex:audit-run-legs-checksum`) con algoritmo SHA-256 y digest explícito; la promoción del checksum a nodo nombrado, en lugar de a blank node, permite referencias externas estables desde sistemas de archivo y logging. Una instancia de `CrossParticipantProvenanceRecord` agrupa la cadena completa abarcando tres `edc:ParticipantContexts`: UPM (proveedor), Leganés (consumidor) e INESData (operador). El equipo del CSIC, como evaluador, publica dos instancias de `it6:Evaluation`: la evaluación de v2 con `accuracy=0.89` y la evaluación de baseline v1 con `accuracy=0.82`; ambas comparten un `SharedEvaluationContext` fijado a `ClimateBench-v1` versión 2026.1, protocolo *holdout* y semilla 42.

5.2. Ciclo publicación-descubrimiento-invocación-trazabilidad-evaluación

El grafo de demostración se utiliza para ejercer las cinco tareas operativas en secuencia. La **publicación** (CQ-R1–CQ-R5) se comprueba consultando el catálogo por proveedor, licencia, política y subpropiedad hacia `foaf:primaryTopic`: las tres ofertas aparecen con su modelo asociado, su política y el contrato de entrada/salida declarado. El **descubrimiento** (CQ-D1–CQ-D4) filtra por tarea, por política y por umbral de calidad. CQ-D1 recupera los tres modelos porque comparten `it6:hasTask`. CQ-D2 devuelve sólo las dos ofertas con política permisiva; la tercera queda filtrada porque su política contiene una prohibición activa sobre `odrl:commercialize`. CQ-D4 añade el filtro por umbral `accuracy ≥ 0,85` sobre el `SharedEvaluationContext` y deja sólo `flood-risk-v2`, que alcanza 0,89.

La **invocación** (CQ-E1–CQ-E5) parte de `offering-flood-v2` y recupera el endpoint, el método de autenticación y el contrato de entrada/salida aprovechando la cadena *subPropertyOf* entre `daimo:offersModel` y `foaf:primaryTopic`. CQ-E2 devuelve `run-2026-04-20-legs` con el municipio como agente asociado y la autorización que la permite. CQ-E3 recupera la algoritmia subyacente, el `it6:Flow` y la infraestructura. CQ-E4 devuelve la evidencia de auditoría completa con el checksum estructurado, el firmante y la marca temporal. CQ-E5 lista los artefactos derivados producidos bajo cada autorización.

La **trazabilidad y evaluación** (CQ-V1–CQ-V5) verifica que el `SharedEvaluationContext` devuelve sus cinco atributos (tarea, dataset, versión, protocolo, semilla), que los dos modelos se ordenan correctamente por métrica dentro del mismo contexto, y que cada evaluación enlaza al `it6:Flow` y a las marcas temporales de ejecución que la respaldan. Finalmente, las **consultas de gobernanza** (CQ-G1–CQ-G4) ejercen exclusivamente semántica DAIMO-nativa: qué ofertas del catálogo federado incluyen un modelo dado, qué despliegues lo sirven con qué infraestructura y contrato, qué autorización permitió una ejecución específica, y cuál es el bundle de procedencia completo para un artefacto derivado —todas preguntas que, como se argumenta en la Sección 7, no pueden responderse con los vocabularios reutilizados aisladamente.

6. Evaluación y validación

DAIMO se evalúa contra las veintitrés preguntas de competencia elicidadas en la fase de especificación y contra las restricciones formales derivadas de la infraestructura axiomática. La evaluación sigue cuatro dimensiones: consistencia lógica y verificación de implicación sobre

el cierre razonado, validación SHACL tanto sobre un grafo positivo como sobre un banco de pruebas deliberadamente inválidas, ejecución de las consultas de competencia sobre el grafo razonado, y las limitaciones de esta evaluación —entre ellas la validación externa con expertos. El paquete público contiene los cuatro scripts (`validate.py`, `reasoner_check.py`, `oops_check.py` y `tests/negative_test.py`) que reproducen íntegramente los reportes; cualquier lector con Python 3.12 y las dependencias declaradas puede ejecutarlos localmente.

6.1. Consistencia lógica y verificación de implicación

El razonador HermiT (invocado mediante `owlready2`) reporta que la unión ontología + grafo de ejemplo es consistente y que no hay clases insatisfactibles. La materialización OWL-RL (ejecutada mediante la biblioteca Python `owlrl`) deriva más de mil tripletas adicionales desde esa misma unión sin producir individuos tipados como `owl:Nothing` ni subclases insatisfactibles. Ambos razonadores coinciden, con HermiT como validador de la corrección lógica y OWL-RL como materializador pragmático para las consultas que requieren inferencia por subpropiedad o subclase.

La consistencia lógica es condición necesaria pero no suficiente para la corrección semántica de una ontología de integración. Una axiomatización puede ser perfectamente consistente y, al mismo tiempo, implicar tipados no intencionados si no declara disyunción explícita con la clase adecuada o si adopta una subpropiedad cuyo rango fuerza una subsunción indeseada. Por eso DAIMO implementa adicionalmente una *verificación de implicación* sobre el cierre OWL-RL: para cada clase nativa, la comprobación enumera todas las superclases inferidas y las contrasta con una lista de superclases prohibidas. Por ejemplo, `AuditEvidence` no debe ser inferida como `prov:Activity`, porque es una cosa *producida por* actividades, no una actividad. El reporte cubre las catorce clases nativas (nueve de primer nivel más cinco subclases de `ParticipantRole`) y no emite ninguna advertencia. Esta comprobación es complementaria a la de consistencia: captura errores de alineamiento que un razonador DL acepta porque son lógicamente consistentes aunque semánticamente incorrectos, y que sólo se manifestarían al consumir el grafo desde una aplicación que asumiera los tipados derivados.

El escáner OOPS! [32], consultado por API REST sobre la unión del núcleo y los alineamientos, reporta cero pitfalls Críticos, cero Importantes y dos Menores: P04 (“elementos no conectados”) afecta únicamente a clases externas declaradas localmente para permitir que el razonador las interprete sin exigir que el archivo las defina completamente, y P13 (“relaciones inversas no declaradas”) afecta a propiedades para las que no existe una inversa semánticamente significativa (p.ej. `daimo:integrityHash`, cuya inversa nombrada no añade utilidad). Ambos Menores son características habituales de perfiles de integración y no indican defectos de modelado.

6.2. Validación SHACL: estructural, invariantes y banco negativo

La validación SHACL se presenta en este apartado integralmente, cubriendo las tres familias de *shapes* —estructurales, de conformidad e invariantes SPARQL— sobre el grafo positivo y el banco de pruebas negativas en un único recorrido, porque su narrativa es una sola.

Las nueve *shapes* estructurales imponen completitud mínima por clase DAIMO-nativa; el grafo positivo de 209 tripletas satisface `conforms=True`. Las tres *shapes* de conformidad añaden obligaciones operativas sobre clases reutilizadas: `OfferInDAIMOShape` exige que toda `odrl:Offer` utilizada en DAIMO declare `odrl:assigner` y `odrl:target` —a nivel de política o por permiso, mediante el conectivo `sh:or`—; `MachineLearningModelInDAIMOShape` exige política ODRL, título e identificador sobre `it6:MachineLearningModel`; `RunInDAIMOShape` exige flow, algoritmo, agente asociado y marca temporal sobre `it6:Run`. Además, `sh:in` restringe los valores de `daimo:authMethod` a un vocabulario cerrado de nueve tokens (`none`, `api-key`, `basic`, `bearer`, `oauth2-bearer`, `oauth2-client-credentials`, `mtls`, `jwt`, `dataspace-token`), y `sh:pattern`

restringe `daimo:protocol` a un patrón que acepta los nombres habituales de protocolo de evaluación (`holdout`, `stratified-holdout`, `leave-one-out`, `temporal-split`, `<n>-fold-cv`, `stratified-<n>-fold-cv` y `bootstrap-<n>`).

Sobre este estrato estructural se sitúan seis invariantes SHACL-SPARQL entre clases, codificadas mediante `sh:SPARQLConstraint`. Son el estrato que distingue a DAIMO de un perfil puramente descriptivo. INV-1 garantiza la consistencia de la cadena de autorización de derivación: todo `DerivedArtifact` debe portar una propiedad `daimo:underAuthorization` cuya autorización cubra precisamente la ejecución de la que el artefacto se deriva. INV-2 impone que el agente asociado a una ejecución mediante `prov:wasAssociatedWith` aparezca también como `daimo:grantedTo` en alguna de las autorizaciones que cubren esa ejecución, haciendo explícito que el actor registrado en PROV y el beneficiario de la autorización ODRL deben ser la misma entidad. INV-3 verifica la coherencia del plano de datos: el `dcat:DataService` expuesto por un `ModelDeployment` debe servir exactamente el modelo que el despliegue declara desplegar, eliminando la posibilidad de que un endpoint anuncie un modelo pero sirva otro. INV-4 exige que el `daimo:expiresAt` de cada `ExecutionAuthorization` sea estrictamente posterior al `prov:startedAtTime` de toda ejecución que la autorización cubre, reflejando la condición elemental de que una autorización no puede justificar ejecuciones que comenzaron tras su caducidad. INV-5 requiere que el modelo referenciado por `daimo:offersModel` en una `AIAssetOffering` aparezca como `odrl:target` de la política asociada, a nivel de política o en alguno de sus permisos; si la política no gobierna el modelo registrado, la cadena de gobernanza está rota aunque ambos recursos existan. INV-6 cierra el mismo triángulo por el lado del proveedor: el `daimo:offeredBy` de toda oferta debe coincidir con el `odrl:assigner` de su política, impidiendo inconsistencias entre la atribución del registro de catálogo y la autoría declarada en la oferta ODRL. El grafo positivo conforma las seis invariantes sin advertencias.

El Listado 3 muestra la codificación concreta de INV-5 como `sh:SPARQLConstraint`. El patrón `FILTER NOT EXISTS` captura la disyunción entre la política a nivel global y el permiso individual: la restricción falla si el modelo ofrecido no aparece como `odrl:target` ni en la política ni en ninguno de sus permisos.

Listing 3: Invariante INV-5 como `sh:SPARQLConstraint`.

```
daimo:OfferingPolicyTargetInvariant a sh:NodeShape ;
  sh:targetClass daimo:AIAssetOffering ;
  sh:sparql [
    a sh:SPARQLConstraint ;
    sh:message "INV-5 violation: offering's offersModel does not
               appear as odrl:target of the attached policy."@en ;
    sh:prefixes daimo:_invariantPrefixes ;
    sh:select """
      SELECT $this ?model ?policy WHERE {
        $this daimo:offersModel ?model ;
        daimo:hasOfferPolicy ?policy .
        FILTER NOT EXISTS { ?policy odrl:target ?model }
        FILTER NOT EXISTS { ?policy odrl:permission/odrl:target ?model }
      }
      """ ;
  ] .
```

Las *shapes* que conforman sobre el grafo positivo son necesarias pero no suficientes para demostrar que las restricciones son semánticamente discriminantes. Una restricción trivialmente satisfacible sobre cualquier grafo pasaría la validación positiva sin aportar garantías reales. Por eso DAIMO publica un grafo complementario `negative-examples.ttl` que codifica seis casos que violan deliberadamente cada invariante sobre nodos focalizados: `bad:INV1-artifact`, `bad:INV2-run`, `bad:INV3-deployment`, `bad:INV4-auth`, `bad:INV5-offering` y `bad:INV6-offering`. El script `negative_test.py` carga la ontología, los *shapes* y el grafo negativo, ejecuta SHACL y

Tabla 7: Cobertura de preguntas de competencia por categoría.

Categoría	Consultas	N.	Inferencia requerida
Registro y publicación (R)	CQ-R1–CQ-R5	5	CQ-R2 (cadena <i>subPropertyOf</i>), CQ-R5 (ruta <i>subClassOf</i>)
Descubrimiento y selección (D)	CQ-D1–CQ-D4	4	CQ-D1 (subtipo de tarea), CQ-D2 (coincidencia de política)
Ejecución y auditabilidad (E)	CQ-E1–CQ-E5	5	CQ-E1 (entailment $\text{offersModel} \sqsubseteq \text{foaf:primaryTopic}$), CQ-E2 (unión <i>agreement-run</i>)
Evaluación y reproducibilidad (V)	CQ-V1–CQ-V5	5	CQ-V2 y CQ-V3 (agregación)
Puente de gobernanza (G)	CQ-G1–CQ-G4	4	CQ-G3 (razonamiento <i>subClassOf</i>), CQ-G4 (agregación <i>GROUP_CONCAT</i>)

verifica dos condiciones: (a) la validación falla globalmente y (b) cada nodo focalizado esperado aparece en el informe con el mensaje específico de la invariante violada. Las seis invariantes se disparan correctamente sobre su violación correspondiente, lo que demuestra que las restricciones SHACL son discriminantes y no simplemente satisfacibles sobre el ejemplo positivo. Esta evidencia complementa a la validación positiva y es la que permite afirmar, con fundamento, que las invariantes capturan reglas de negocio reales.

6.3. Preguntas de competencia

Las veintitrés preguntas de competencia se expresan como consultas SPARQL y se ejecutan sobre el cierre OWL-RL materializado del grafo positivo. La Tabla 7 resume la cobertura por categoría y destaca las consultas que requieren razonamiento explícito; el conjunto completo de consultas aparece en el Apéndice C. Las veintitrés devuelven al menos una fila, lo que verifica que el grafo de demostración contiene la información necesaria para responder a cada pregunta con las primitivas que DAIMO declara.

Un punto merece ser señalado explícitamente: dos consultas —CQ-R2 y CQ-E1— dependen de la relación *subPropertyOf* entre `daimo:offersModel` y `foaf:primaryTopic` y no devuelven filas sobre un motor SPARQL puro sin razonamiento. Esta dependencia es documentada y no un defecto: la decisión de alinear `offersModel` como subpropiedad de `foaf:primaryTopic` proporciona beneficios semánticos (las ofertas son descubribles mediante la propiedad DCAT estándar) que sólo se materializan bajo inferencia. Un consumidor que necesite ejecutar estas consultas sobre un endpoint SPARQL puro puede optar por materializar el cierre previamente, o reescribir las consultas para enumerar ambas propiedades.

6.4. Limitaciones de la evaluación actual

La evaluación presentada es íntegramente formal y ejecutable, y su naturaleza acota con precisión lo que puede afirmarse sobre DAIMO. Dos limitaciones requieren declaración explícita. La primera es que las preguntas de competencia, el grafo positivo, el banco negativo y las consultas SPARQL han sido contruidos por los autores: la validación demuestra consistencia interna y corrección formal, no garantiza que las preguntas capturen las necesidades reales de un practicante externo ni que el grafo refleje patrones que emergen en despliegues operativos. La segunda es que no se ha realizado aún una evaluación con expertos del dominio. Un estudio cualitativo con practicantes está planificado y, cuando se ejecute, involucrará al menos tres perfiles complementarios: un ingeniero de conectores EDC o de infraestructura INESData, un mantenedor de MLDCAT-AP o contacto SEMIC, y un practicante MLOps con responsabilidades sobre catálogos internos de modelos. El objetivo será evaluar si la ontología captura el nivel de abstracción adecuado para el trabajo diario, si los compromisos del perfil resultan prácticos

de mantener, y si la información exigida por las preguntas de competencia puede efectivamente producirse y actualizarse en el flujo operativo real. Este estudio se marca explícitamente como **[PENDIENTE: Ejecutar estudio con expertos: reclutamiento, protocolo de entrevista semiestructurada, análisis cualitativo y publicación de resultados. Los tres perfiles objetivo están identificados.]**

7. Discusión

DAIMO articula publicación, política, ejecución, procedencia y evaluación dentro de un único perfil de integración diseñado para soportar el intercambio gobernado de modelos a través de fronteras organizativas. Esta posición lo diferencia tanto de ontologías de dominio publicadas recientemente —RePlanIT [9], GloSIS [10] o la ontología de mantenimiento [11]— como de las ontologías previas del ciclo de vida de aprendizaje automático. Con las primeras comparte la ética de ingeniería ontológica —reutilización disciplinada, implementación ejecutable y evaluación contra preguntas de competencia—, pero la aplica al puente específico entre catálogo de IA y plano de datos. Con las segundas comparte el dominio de aplicación, pero suma la capa de gobernanza que ellas no abordan. Tres rasgos de diseño diferencian el artefacto. El primero es la verificación de implicación sobre el cierre OWL-RL, que complementa al razonamiento de consistencia enumerando las superclases inferidas de cada clase nativa y detectando así alineamientos semánticamente incorrectos que un razonador DL acepta sin contradicción. El segundo son las invariantes SHACL-SPARQL entre clases, acompañadas de un banco de pruebas negativas que confirma que las restricciones son discriminantes: cada invariante se dispara sobre un caso diseñado para violarla, no solamente sobre el grafo positivo. El tercero es la orientación *reuse-first* encarnada en alineamientos axiomáticos explícitos, no simples listas de prefijos, y en la decisión documentada de no declarar tres subpropiedades hacia PROV-O cuya adopción habría producido tipados no intencionados sobre clases de artefacto.

Las limitaciones honestas del artefacto pueden agruparse en tres líneas argumentales. La primera concierne al *nivel de detalle semántico* de varias extensiones DAIMO. El `IOContract` captura el formato de entrada/salida y el método de autenticación —este último restringido por `sh:in` a un vocabulario cerrado— pero no ofrece compatibilidad semántica fina entre los esquemas del modelo y los del consumidor, remitiendo esas dimensiones a `it6:Flow`, como ya hacen Kipoi [22] y TFX [23] para los aspectos de preprocesamiento y *pipelines*. El `SharedEvaluationContext` reifica la combinación tarea-dataset-versión-protocolo-semilla suficiente para comparaciones homogéneas sobre una métrica única, pero no cubre protocolos heterogéneos de *benchmarking* ni conjuntos compuestos de métricas. El modelado de roles incurre en una confluencia pragmática entre tipo de rol (`ModelProvider`) y asignación de rol —la tupla agente-rol-contexto-alcance— que OntoClean recomienda separar, pero que MLDCAT-AP también adopta en `hasRegisteredUser`, indicando que la contextualización fina es un refinamiento futuro razonable antes que un error de diseño.

La segunda línea concierne a los *compromisos de integración* y a los efectos que esos compromisos tienen sobre consumidores que combinen DAIMO con vocabularios adyacentes. La propiedad `daimo:records` declara `rdfs:range prov:Activity`, de modo que bajo razonamiento RDFS las evaluaciones MLDCAT-AP agrupadas dentro de un `CrossParticipantProvenanceRecord` son inferidas como actividades PROV aunque MLDCAT-AP no declare esa subsunción; el efecto es inocuo para consultas de auditoría, pero un consumidor que integre DAIMO, MLDCAT-AP y PROV-O puede obtener tipados sorprendentes sobre `it6:Evaluation`. En la misma línea, DAIMO no se ancla a una ontología de nivel superior (BFO, GFO, DOLCE o UFO); la ausencia es deliberada porque un perfil de integración no es una ontología de referencia, pero un trabajo futuro coherente sería acomodar la taxonomía a una de esas ontologías superiores sin renunciar a la reutilización directa de los vocabularios de dominio que motivan DAIMO.

La tercera línea concierne a las *condiciones de reproducibilidad y validación externa*. Los

resultados numéricos reportados dependen de un perfil de razonamiento concreto: el modo `inference=rdfs` en `pyshacl` —necesario para que las subpropiedades implicadas se materialicen antes de evaluar los *shapes* de conformidad— y el cierre OWL-RL materializado antes de las consultas de competencia. Esta dependencia está documentada en el paquete, pero es una condición que un consumidor debe conocer para reproducir los resultados exactos; las consultas que dependen de *subPropertyOf* se detallan en §6.2. Adicionalmente, como ya se ha indicado, la validación con expertos del dominio no forma parte de la evidencia reportada: constituye el siguiente paso natural del trabajo, pero su ausencia acota las garantías que un lector puede extraer del artículo actual.

La relación entre DAIMO y MLDCAT-AP merece explicitarse para disipar cualquier preocupación de duplicidad. El modelo propiamente dicho se reutiliza directamente a través de `it6:MachineLearningModel` y no es redefinido. La contribución específica se concentra en el puente entre catálogo y plano de datos: las cuatro preguntas de competencia de la categoría de gobernanza ejercitan exclusivamente clases nativas DAIMO y no pueden responderse sólo con MLDCAT-AP, porque requieren `AIAssetOffering`, `ModelDeployment`, `ExecutionAuthorization` y `CrossParticipantProvenanceRecord`, respectivamente. Esta delimitación operativa es la evidencia concreta de que la contribución no es redundante sino complementaria, y de que la elección de reutilizar MLDCAT-AP en lugar de duplicar su vocabulario es coherente con la disciplina *reuse-first* declarada desde el principio.

8. Disponibilidad y sostenibilidad

El artefacto DAIMO se distribuye como un paquete público bajo licencia CC-BY 4.0 para la ontología y su documentación, y Apache-2.0 para los scripts de validación. La Tabla 8 resume los elementos entregables y sus ubicaciones. El runbook operativo para la publicación está documentado en `DEPLOYMENT.md`, de modo que el procedimiento de liberación es ejecutable por cualquier mantenedor futuro sin conocimiento tácito adicional. Los elementos en rojo indican acciones de publicación aún pendientes de ejecución.

El flujo de reproducibilidad no requiere el despliegue previo de la URL persistente: cualquier usuario con Python 3.12 y las dependencias declaradas (`rdflib`, `pyshacl`, `owlrl`, `owlready2`) puede ejecutar los cuatro scripts localmente y reproducir íntegramente los reportes de `reports/`. La URL persistente y el DOI son requisitos FAIR adicionales que complementan pero no condicionan la reproducibilidad. La cabecera de la ontología declara `dct:conformsTo` hacia OWL 2 y utiliza `rdfs:seeAlso` para enlazar los tres módulos y la documentación HTML, de modo que un consumidor puede recorrer la cadena identificador-documento sin cargar el paquete completo.

La sostenibilidad del artefacto se estructura según la Fase 4 de LOT. `CONTRIBUTING.md` documenta el flujo de triaje de *issues*, el ciclo de pull requests y la política de versionado según SemVer: las versiones *patch* se reservan para correcciones de definición textual o documentación, las *minor* para nuevas clases o propiedades compatibles, y las *major* para cambios que alteran alineamientos o invariantes. La política de deprecación impone al menos una versión *minor* de coexistencia antes de retirar un término, con `owl:deprecated` y `skos:closeMatch` hacia el término de reemplazo.

9. Conclusiones

DAIMO conecta MLDCAT-AP, DCAT, ODRL, PROV-O y el protocolo DSP para convertir los modelos de IA en activos gobernables de primera clase dentro de un espacio de datos. Su contribución específica es la capa de gobernanza ausente entre el catálogo de IA y el plano de datos —registro con política ODRL coherente, autorización de ejecución anclada a un acuerdo, despliegue como puente tipado entre el modelo y su servicio, y procedencia agregada entre contextos administrativos—, capa que los vocabularios reutilizados aisladamente no proporcionan

Tabla 8: Disponibilidad del artefacto DAIMO como paquete público reproducible.

Elemento	Ubicación
Namespace persistente	https://w3id.org/pionera/daimo [PENDIENTE: Ejecutar merge del PR sobre perma-id/w3id.org para que el namespace resuelva por content negotiation.]
Módulos versionados	Tres ontologías independientes (núcleo, <i>shapes</i> , alineamientos) bajo https://w3id.org/pionera/daimo/ , con owl:versionIRI y owl:priorVersion
Repositorio público	[PENDIENTE: Crear repositorio GitHub público pionera/daimo (ejecución del runbook DEPLOYMENT.md) e incluir la URL definitiva aquí.]
Archivado versionado (DOI)	[PENDIENTE: Archivar la primera release en Zenodo vía la integración GitHub → Zenodo e incluir el DOI resultante aquí.]
Serializaciones	Turtle, RDF/XML, JSON-LD, N-Triples (pre-generadas por WIDOCO)
Documentación HTML	WIDOCO 1.4.25 con WebVOWL integrado; negociación de contenido vía .htaccess bajo w3id-redirect/
Referencia humana-legible	ONTOLOGY-REFERENCE.md (clase por clase, con skos:definition, skos:example, criterios de identidad y etiquetas OntoClean) y VALIDATION-MATRIX.md (trazabilidad requisito-evidencia)
Metadatos de <i>release</i>	CITATION.cff, .zenodo.json y CHANGELOG.md
Paquete de validación	validate.py, reasoner_check.py, oops_check.py, tests/negative_test.py
Gobernanza de mantenimiento	CONTRIBUTING.md (Fase 4 LOT: triaje de <i>issues</i> , pull requests, SemVer y política de deprecación)

y que cuatro de las veintitrés preguntas de competencia permiten verificar de forma concreta. El caso de estudio multi-participante ilustra que la ontología puede instanciarse coherentemente sobre un escenario verosímil y responder las cinco familias de consultas operativas que un espacio de datos gobernado plantea. Las líneas de trabajo futuro se derivan directamente de las limitaciones acotadas: instanciación contra catálogos MLDCAT-AP reales, evaluación cualitativa con expertos del dominio, refinamiento del IOContract hacia compatibilidad semántica de esquemas más allá del formato y la autenticación, separación rol-tipo/rol-asignación siguiendo el patrón OntoClean, y extensión del SharedEvaluationContext a protocolos heterogéneos de *benchmarking*. El artefacto se distribuye como paquete público con namespace persistente, documentación WIDOCO, capa SHACL independiente y suite de validación reproducible, preparado para ser adoptado, extendido o criticado por la comunidad con la misma disciplina metodológica con la que ha sido construido.

Referencias

- [1] M. Franklin, A. Halevy, and D. Maier. From Databases to Dataspace: A New Abstraction for Information Management. *SIGMOD Record*, 34(4):27–33, 2005.
- [2] A. Albertoni et al. Data Catalog Vocabulary (DCAT) – Version 2. W3C Recommendation, 2020.
- [3] P. Lebo, S. Sahoo, and D. McGuinness, editors. *PROV-O: The PROV Ontology*. W3C Recommendation, 2013.
- [4] R. Iannella, S. Guth, and R. Steyskal, editors. *ODRL Information Model 2.2*. W3C Recommendation, 2018.

- [5] H. Knublauch and D. Kontokostas, editors. *Shapes Constraint Language (SHACL)*. W3C Recommendation, 2017.
- [6] M. Mitchell et al. Model Cards for Model Reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 2019.
- [7] M. Arnold et al. FactSheets: Increasing Trust in AI Services through Supplier’s Declarations of Conformity. IBM Research Report, 2019.
- [8] J. Werbrouck et al. ConSolid: A federated ecosystem for heterogeneous multi-stakeholder projects. *Semantic Web*, 15:429–460, 2024.
- [9] A. Kurteva et al. RePlanIT Ontology for Digital Product Passports of ICT: Laptops and Data Servers. *Semantic Web*, in press.
- [10] R. Palma et al. GloSIS: The Global Soil Information System Web Ontology. *Semantic Web*, in press.
- [11] C. Woods et al. An ontology for maintenance activities and its application to data quality. *Semantic Web*, in press.
- [12] B.E. Penteado, J.C. Maldonado, and S. Isotani. Methodologies for publishing linked open government data on the Web: A systematic mapping and a unified process model. *Semantic Web*, 14:585–610, 2023.
- [13] Y.S. Bareedu et al. Deriving semantic validation rules from industrial standards: An OPC UA study. *Semantic Web*, 15:517–554, 2024.
- [14] N. Ferranti et al. Formalizing and Validating Wikidata’s Property Constraints using SHACL and SPARQL. *Semantic Web*, in press.
- [15] H.J. Pandit and B. Esteves. Enhancing Data Use Ontology (DUO) for Health-Data Sharing by Extending it with ODRL and DPV. *Semantic Web*, in press.
- [16] L. Robaldo and S. Batsakis. On the interplay between validation and inference in SHACL: an investigation on the Time Ontology. *Semantic Web*, in press.
- [17] B. Otto et al. Reference architecture model for international data spaces. *International Journal of Information Management*, 45:182–199, 2019.
- [18] C. Cappiello, A. Gal, M. Jarke, J. Rehof, and Y. Yang. Data spaces: Design, deployment, and future directions. *ACM Computing Surveys*, 55(2):Article 46, 2022.
- [19] T. Gebru et al. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.
- [20] J. Vanschoren, J.N. van Rijn, B. Bischl, and L. Torgo. OpenML: Networked science in machine learning. *SIGKDD Explorations*, 15(2):49–60, 2014.
- [21] M. Vartak et al. ModelDB: A system for machine learning model management. In *Proceedings of the Workshop on Human-in-the-Loop Data Analytics*, 2016.
- [22] Z. Avsec et al. The Kipoi repository accelerates community exchange and reuse of predictive models for genomics. *Nature Biotechnology*, 37, 2019.
- [23] D. Baylor et al. TFX: A TensorFlow-based production-scale machine learning platform. In *KDD*, 2017.
- [24] L.N. Soldatova and R.D. King. An ontology of scientific experiments. *Journal of the Royal Society Interface*, 3(11), 2006.

- [25] G. Correa Publio et al. ML-Schema: Exposing the semantics of machine learning with schemas and ontologies. 2018.
- [26] C.M. Keet, J.D. Fernández, and P. Panov. MEX vocabulary: A lightweight interchange format for machine learning experiments. In *ISWC*, 2016.
- [27] P. Panov, S. Dzeroski, and L.N. Soldatova. OntoDM: An ontology of data mining. In *ICDMW*, 2008.
- [28] C.M. Keet et al. The data mining optimization ontology. *Web Semantics: Science, Services and Agents on the WWW*, 2015.
- [29] R. Souza et al. Provenance data in the machine learning lifecycle in computational science and engineering. In *WORKS*, 2019.
- [30] M. Schmidt et al. FAIRness along the machine learning lifecycle using Dataverse in combination with MLflow. In *ICDE Workshops*, 2021.
- [31] M. Poveda-Villalón, A. Fernández-Izquierdo, M. Fernández-López, and R. García-Castro. LOT: An industrial oriented ontology engineering framework. *Engineering Applications of Artificial Intelligence*, 111:104755, 2022.
- [32] M. Poveda-Villalón, A. Gómez-Pérez, and M.C. Suárez-Figueroa. OOPS! (OntOlogy Pitfall Scanner!). *International Journal on Semantic Web and Information Systems*, 10(2):7–34, 2014.
- [33] SEMIC. *MLDCAT-AP 3.0.0: Machine Learning Data Catalogue Application Profile*. European Commission, 2025. <https://semiceu.github.io/MLDCAT-AP/releases/3.0.0/>.
- [34] International Data Spaces Association. *Dataspace Protocol (DSP)*. W3C Community Draft, 2024.
- [35] Eclipse Foundation. *Eclipse Dataspace Components (EDC)*, control-plane documentation. <https://eclipse-edc.github.io/documentation/>.
- [36] D. Garijo. WIDOCO: A Wizard for Documenting Ontologies. In *ISWC*, 2017.
- [37] European Union. *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act)*. Official Journal of the European Union, 2024.

A. Preguntas de competencia

Registro y publicación (5).

1. CQ-R1: ¿Qué modelos ha publicado un proveedor dado como activos de catálogo gobernados?
2. CQ-R2: Dado un modelo, ¿qué oferta lo registró, por qué publicador y en qué fecha? (requiere cadena `subPropertyOf`)
3. CQ-R3: ¿Qué licencia y política de uso aplican a un modelo publicado?
4. CQ-R4: ¿Qué contrato de entrada/salida se declara para la interfaz de invocación?
5. CQ-R5: Para cada oferta, ¿qué subclase concreta de `ParticipantRole` tiene el agente proveedor? (requiere `subClassOf+`)

Descubrimiento y selección (4).

1. CQ-D1: ¿Qué modelos resuelven una tarea dada en un dominio dado?
2. CQ-D2: ¿Qué modelos son utilizables bajo un patrón de política, p. ej. uso no comercial?
3. CQ-D3: ¿Qué modelos exponen un endpoint y qué método de autenticación requiere cada uno?
4. CQ-D4: ¿Qué modelos alcanzan un umbral mínimo de métrica bajo un contexto de evaluación dado?

Ejecución y auditabilidad (5).

1. CQ-E1: Dada una oferta, ¿qué endpoint, método de autenticación y contrato de entrada/salida aplican al modelo subyacente? (requiere *entailment*)
2. CQ-E2: Para un despliegue dado, ¿qué ejecuciones, por qué agentes y bajo qué autorización?
3. CQ-E3: Para una ejecución dada, ¿qué implementación, algoritmo, *flow* e infraestructura?
4. CQ-E4: ¿Qué evidencia de auditoría (hash, firmante, marca temporal) respalda una ejecución?
5. CQ-E5: ¿Qué artefactos derivados produjo una ejecución y bajo qué autorización?

Evaluación y reproducibilidad (5).

1. CQ-V1: ¿Qué contexto compartido (dataset, versión, protocolo, semilla) usa una evaluación?
2. CQ-V2: Bajo un contexto compartido, ¿qué modelo alcanza el mayor valor de métrica?
3. CQ-V3: ¿Cómo se ordenan dos o más modelos bajo el mismo contexto y métrica?
4. CQ-V4: Para un modelo dado, ¿sobre qué benchmarks ha sido evaluado?
5. CQ-V5: ¿Qué artefactos de reproducibilidad (flow, cuaderno, tabla de resultados, checksum) respaldan una evaluación?

Puente de gobernanza (4, DAIMO-nativas).

1. CQ-G1: ¿Qué ofertas del catálogo federado incluyen un modelo dado?
2. CQ-G2: ¿Qué despliegues sirven un modelo dado, sobre qué infraestructura y con qué contrato de entrada/salida?
3. CQ-G3: ¿Qué autorización (y el acuerdo del que deriva) permitió una ejecución específica?
4. CQ-G4: A través de contextos de participante, ¿cuál es el bundle de procedencia completo para un artefacto derivado?

B. Shapes SHACL

La capa SHACL de DAIMO contiene nueve *shapes* nodales de completitud, tres *shapes* de conformidad sobre clases reutilizadas y seis invariantes SHACL-SPARQL entre clases. El módulo `shapes/daimo-shapes.ttl` se declara a su vez como `owl:Ontology` independiente con su propia `owl:versionIRI` bajo <https://w3id.org/pionera/daimo/shapes/>, creador, licencia y metadatos de publicación, de modo que la capa de validación se libera y se referencia con ciclo de vida propio.

Shapes de completitud (9). `AIAssetOfferingShape`, `ParticipantRoleShape`, `ModelDeploymentShape`, `IOContractShape`, `ExecutionAuthorizationShape`, `DerivedArtifactShape`, `SharedEvaluationContextShape`, `AuditEvidenceShape` y `CrossParticipantProvenanceRecordShape`.

Shapes de conformidad sobre clases reutilizadas (3). `OfferInDAIMOShape` exige `odrl:assigner` y `odrl:target` sobre toda `odrl:Offer`; `MachineLearningModelInDAIMOShape` exige política, título e identificador sobre `it6:MachineLearningModel`; `RunInDAIMOShape` exige *flow*, algoritmo, agente asociado y marca temporal sobre `it6:Run`.

Invariantes cruzadas (6). `DerivationAuthorizationInvariant` (INV-1), `RunAgentAuthorizationInvariant` (INV-2), `DeploymentServiceInvariant` (INV-3), `AuthorizationTemporalInvariant` (INV-4), `OfferingPolicyTargetInvariant` (INV-5) y `OfferingAssignerInvariant` (INV-6). Las seis se declaran como `sh:SPARQLConstraint` y comparten el bloque de prefijos `daimo:_invariantPrefixes`.

C. Consultas SPARQL

Las veintitrés consultas SPARQL correspondientes a las preguntas de competencia del Apéndice A están publicadas en `queries/queries.md`. El validador ejecuta cada consulta sobre el cierre OWL-RL del grafo unión ontología + ejemplo y verifica que cada una devuelve al menos una fila. CQ-G2 devuelve dos filas en el grafo de demostración —una por cada endpoint del despliegue multi-protocolo REST y gRPC—, lo que confirma que el modelado de múltiples servicios por despliegue es un patrón legítimo y observable desde una consulta SPARQL directa.

D. Convenciones de URI

DAIMO utiliza el namespace canónico <https://w3id.org/pionera/daimo>, con identificadores de términos en la forma <https://w3id.org/pionera/daimo#<Término>> e IRIs versionadas bajo <https://w3id.org/pionera/daimo/<versión>>, enlazadas entre sí mediante `owl:priorVersion`. Los módulos satélite son ontologías independientes con sus propias IRIs versionadas: la capa SHACL bajo <https://w3id.org/pionera/daimo/shapes/> y el módulo de alineamientos bajo <https://w3id.org/pionera/daimo/align>, este último declarando `owl:imports` sobre la ontología base. El grafo de ejemplo utiliza el namespace local <https://example.org/daimo-scenario/>, claramente separado del espacio canónico para evitar que los identificadores de demostración se confundan con términos de la ontología. La negociación de contenido resuelve el mismo identificador a HTML, Turtle, RDF/XML, JSON-LD o N-Triples en función del `Accept` header; la configuración `.htaccess` correspondiente está preparada en `w3id-redirect/.htaccess` a la espera de su integración en `perma-id/w3id.org`.